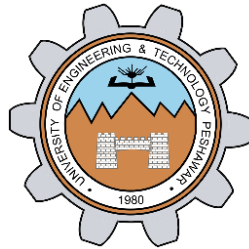


BCD to Seven Segment Decoder

LAB # 05



Spring 2024

CSE-308L Digital System Design Lab

Submitted by: **Ali Asghar**

Registration No.: **21PWCSE2059**

Class Section: **C**

“On my honor, as student of University of Engineering and Technology, I have neither given nor received unauthorized assistance on this academic work.”

Submitted to:

Dr. Asif Ali Khan

Date:

29th March 2024

Department of Computer Systems Engineering
University of Engineering and Technology, Peshawar

Objectives:

This lab will enable students to:

- To implement a BCD to Seven Segment Decoder on Spartan 6 board.

Software used:

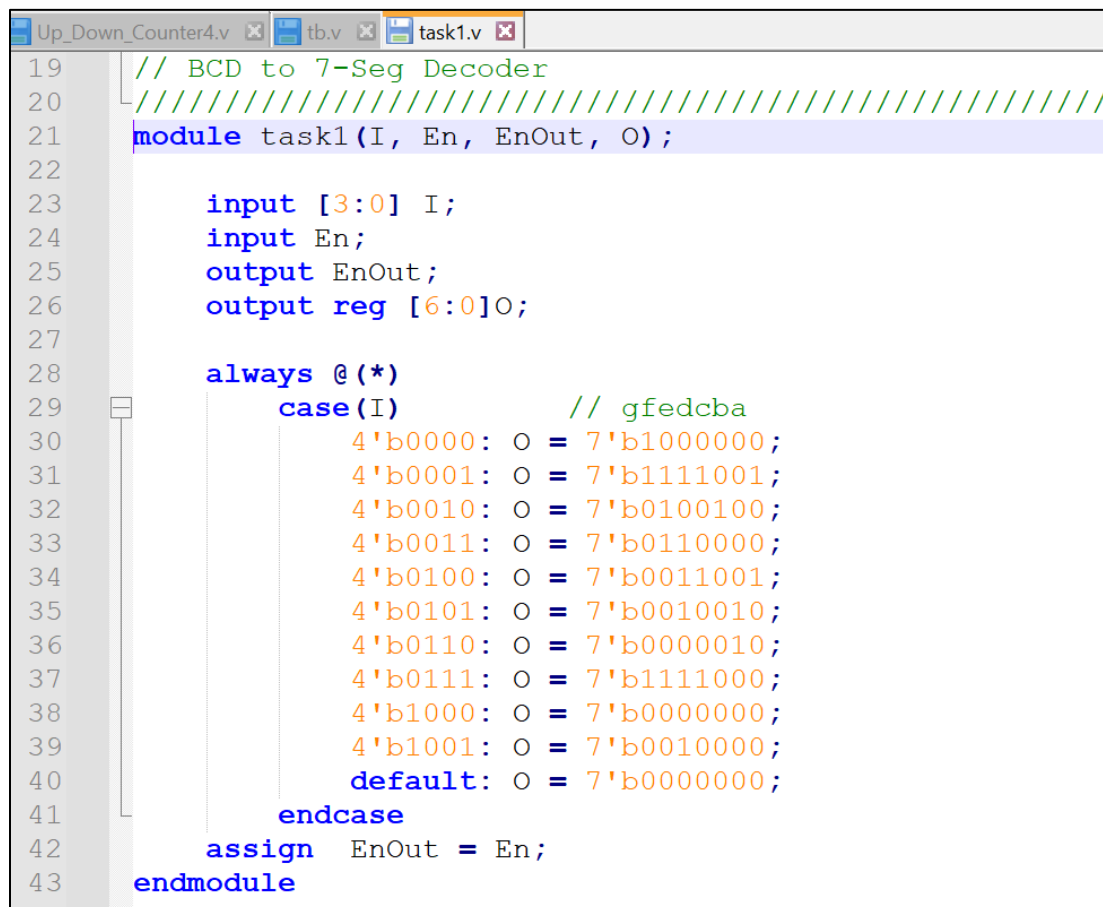
- Xilinx ISE

Task 1:

Using switches enter a BCD number and show the resulting number on the seven segment display.

Code:

DUT:

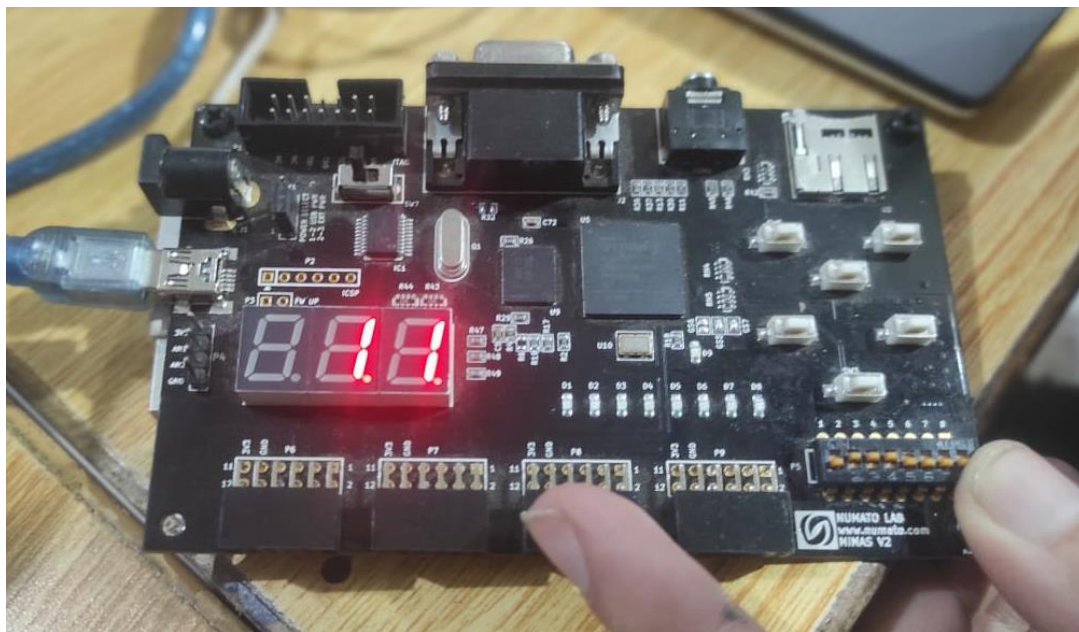
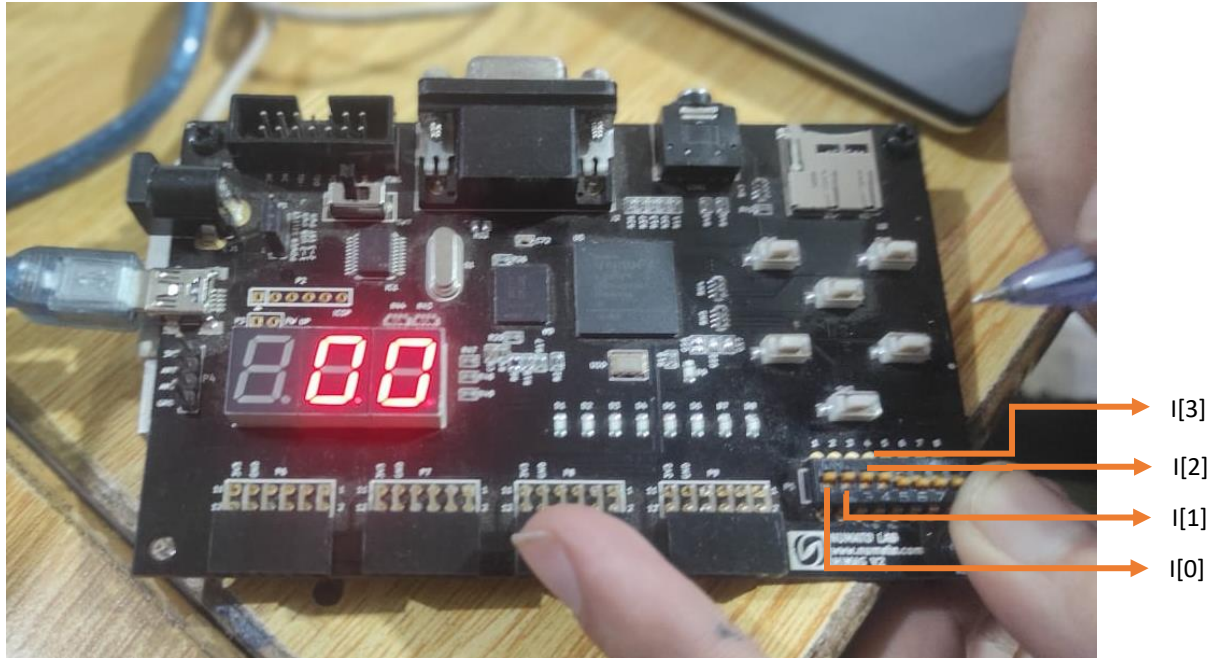


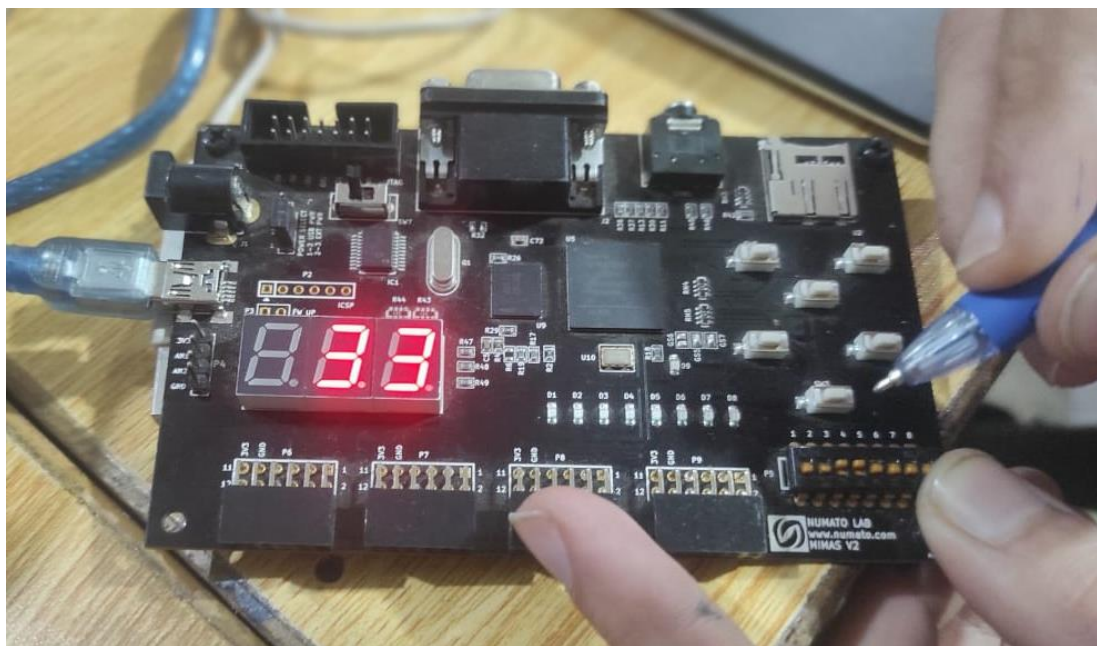
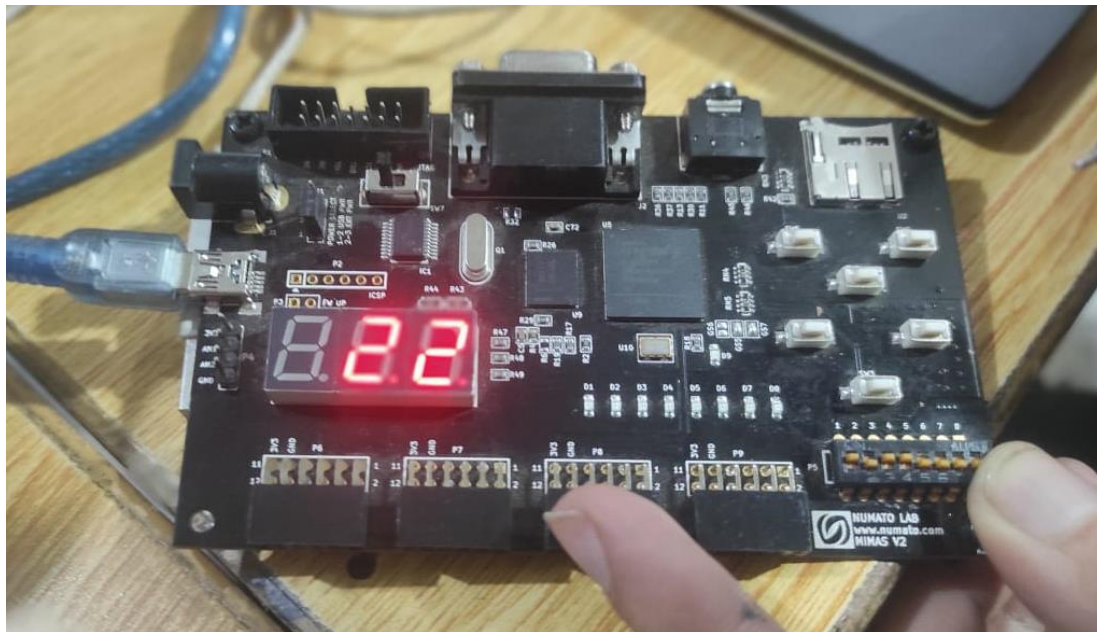
```
19 // BCD to 7-Seg Decoder
20 //////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////
21 module task1(I, En, EnOut, O);
22
23     input [3:0] I;
24     input En;
25     output EnOut;
26     output reg [6:0] O;
27
28     always @(*)
29     case(I) // gfedcba
30         4'b0000: O = 7'b1000000;
31         4'b0001: O = 7'b1111001;
32         4'b0010: O = 7'b0100100;
33         4'b0011: O = 7'b0110000;
34         4'b0100: O = 7'b0011001;
35         4'b0101: O = 7'b0010010;
36         4'b0110: O = 7'b0000010;
37         4'b0111: O = 7'b1111000;
38         4'b1000: O = 7'b0000000;
39         4'b1001: O = 7'b0010000;
40         default: O = 7'b0000000;
41     endcase
42     assign EnOut = En;
43 endmodule
44
```

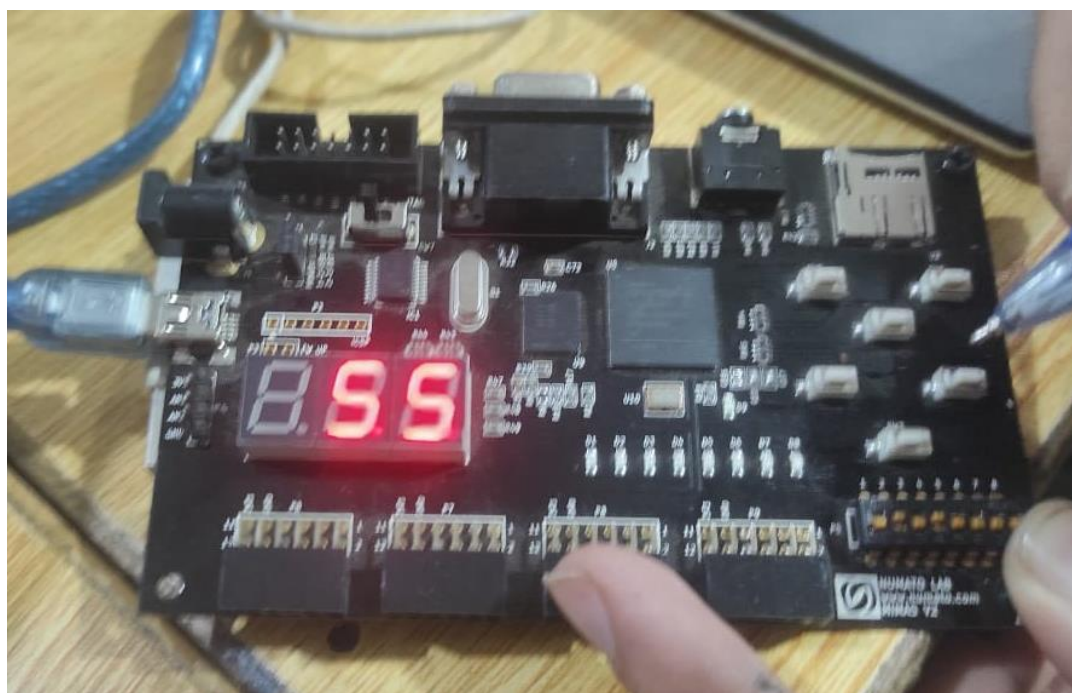
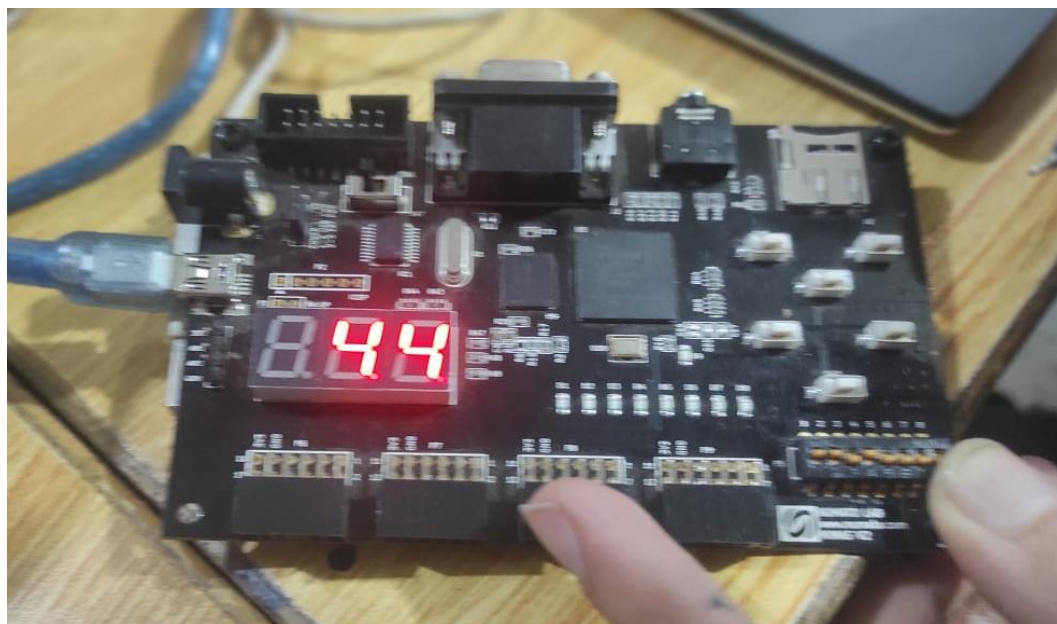
ucf file:

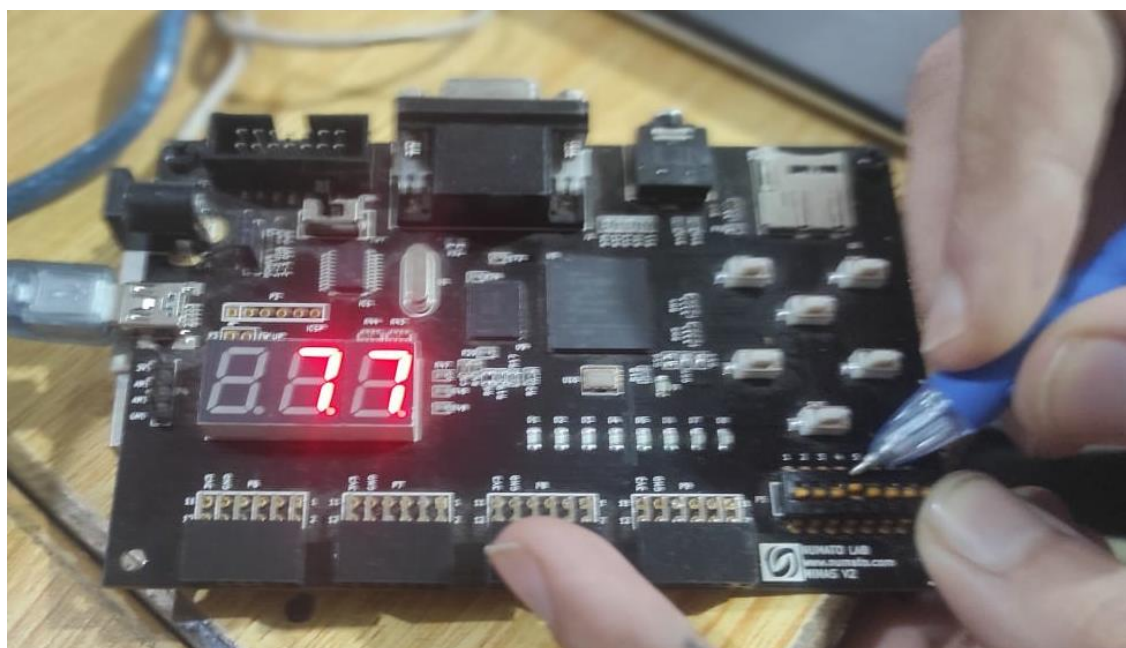
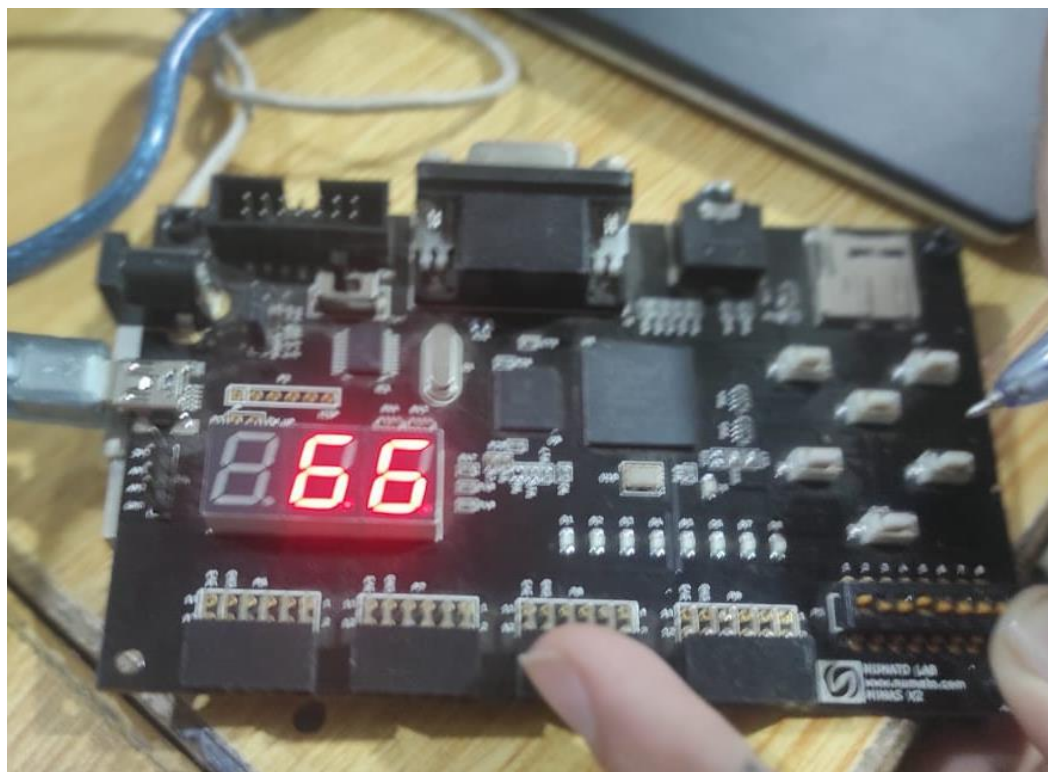
```
Up_Down_Counter4.v x tb.v x task1.v x task1.ucf x
1  # PlanAhead Generated physical constraints
2
3  NET "I[0]" LOC = F17;
4  NET "I[1]" LOC = F18;
5  NET "I[2]" LOC = E16;
6  NET "I[3]" LOC = E18;
7  NET "En" LOC = C17;
8
9  NET "O[0]" LOC = A3;
10 NET "O[1]" LOC = B4;
11 NET "O[2]" LOC = A4;
12 NET "O[3]" LOC = C4;
13 NET "O[4]" LOC = C5;
14 NET "O[5]" LOC = D6;
15 NET "O[6]" LOC = C6;
16
17 NET "EnOut" LOC = B3;
18 # PlanAhead Generated IO constraints
19
20 NET "I[0]" PULLUP;
21 NET "I[1]" PULLUP;
22 NET "I[2]" PULLUP;
23 NET "I[3]" PULLUP;
24 NET "En" PULLUP;
25
26 NET "O[0]" SLEW = FAST;
27 NET "O[1]" SLEW = FAST;
28 NET "O[2]" SLEW = FAST;
29 NET "O[3]" SLEW = FAST;
30 NET "O[4]" SLEW = FAST;
31 NET "O[5]" SLEW = FAST;
32 NET "O[6]" SLEW = FAST;
33
34 NET "EnOut" SLEW = FAST;
```

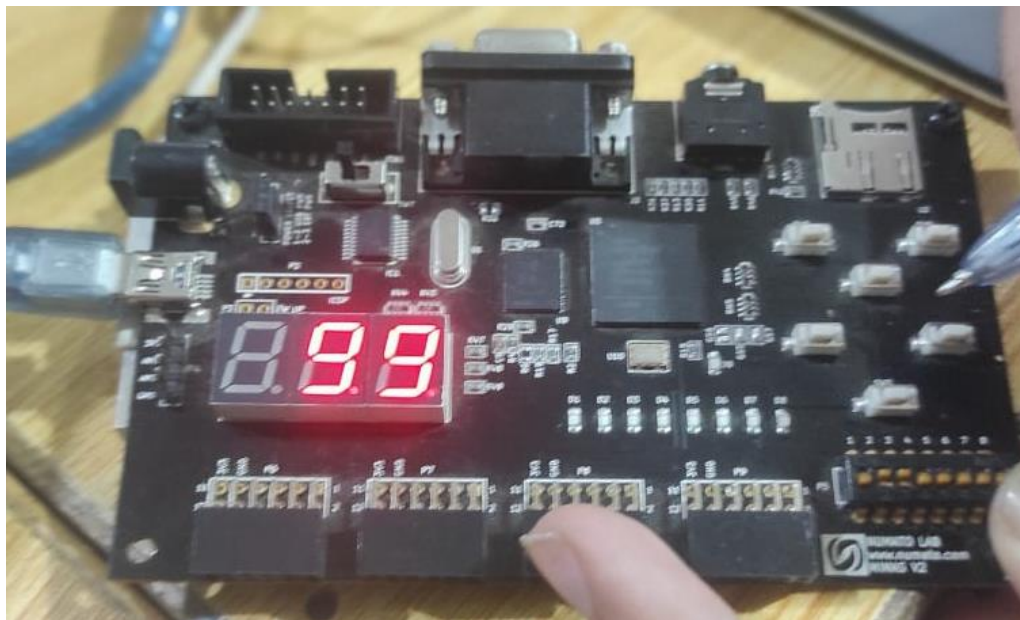
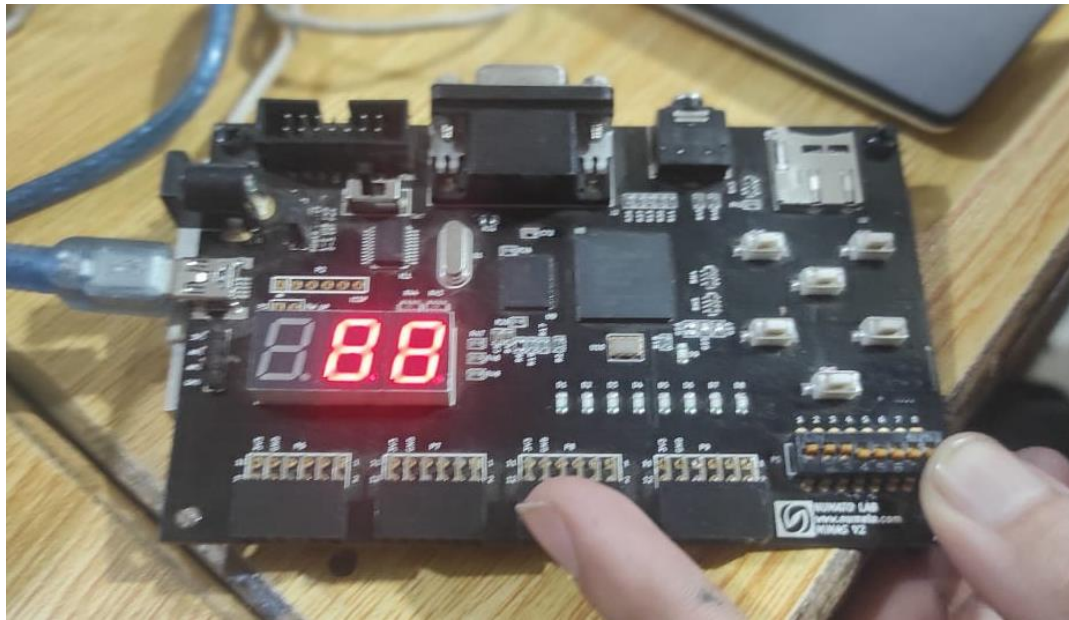
Output:











Task 2:

Connect the output of your lab 02 (4 bit adder) to the seven segment display. Note that number above 1001 are not valid BCD numbers. In this situation keep the seven segment display off and just the dp on.

Code:

DUT:

The screenshot shows the Xilinx IDE interface. On the left, the 'Hierarchy' pane displays the project structure: Task2, xc6s1x9-2csg324, task2 (task2.v), and task2.ucf. Below it, the 'Processes: task2' pane shows a list of steps: Design Summary/Reports, Design Utilities, User Constraints, Synthesize - XST, Implement Design, Generate Programming File (highlighted in blue), Configure Target Device, and Analyze Design Using ChipScope. The main editor on the right displays the Verilog code for the task2 module.

```
21 module task2 (O, a, b);
22     input [3:0] a, b;
23     output reg [6:0] O;
24     wire [3:0] s;
25
26     assign s = a + b;
27     always @(*)
28
29         case(s)           // gfedcba
30             4'b0000: O = 7'b1000000;
31             4'b0001: O = 7'b1111001;
32             4'b0010: O = 7'b0100100;
33             4'b0011: O = 7'b0110000;
34             4'b0100: O = 7'b0011001;
35             4'b0101: O = 7'b0010010;
36             4'b0110: O = 7'b0000010;
37             4'b0111: O = 7'b1111000;
38             4'b1000: O = 7'b0000000;
39             4'b1001: O = 7'b0010000;
40             default: O = 7'b0000000;
41             // 4'h0: O = 7'h40;
42             // 4'h1: O = 7'h79;
43             // 4'h2: O = 7'h24;
```

ucf file:

The screenshot shows the task2.ucf file in the Xilinx IDE. The file contains PlanAhead generated physical and IO constraints. The physical constraints section defines the location (LOC) for signals a[0], a[1], a[2], a[3], b[0], b[1], b[2], b[3], O[0], O[1], O[2], O[3], O[4], O[5], and O[6]. The IO constraints section defines the pullup configuration for signals a[0], a[1], a[2], and a[3].

```
1 # PlanAhead Generated physical constraints
2 NET "a[0]" LOC = F17;
3 NET "a[1]" LOC = F18;
4 NET "a[2]" LOC = E16;
5 NET "a[3]" LOC = E18;
6
7 NET "b[0]" LOC = D18;
8 NET "b[1]" LOC = D17;
9 NET "b[2]" LOC = C18;
10 NET "b[3]" LOC = C17;
11
12 NET "O[0]" LOC = A3;
13 NET "O[1]" LOC = B4;
14 NET "O[2]" LOC = A4;
15 NET "O[3]" LOC = C4;
16 NET "O[4]" LOC = C5;
17 NET "O[5]" LOC = D6;
18 NET "O[6]" LOC = C6;
19
20 # PlanAhead Generated IO constraints
21 NET "a[0]" PULLUP;
22 NET "a[1]" PULLUP;
23 NET "a[2]" PULLUP;
24 NET "a[3]" PULLUP;
25
```

The task2.ucf file is open in the IDE, and the Design Summary (running) window is visible at the bottom.

```

25
26 NET "b[0]" PULLUP;
27 NET "b[1]" PULLUP;
28 NET "b[2]" PULLUP;
29 NET "b[3]" PULLUP;
30
31 NET "O[0]" SLEW = FAST;
32 NET "O[1]" SLEW = FAST;
33 NET "O[2]" SLEW = FAST;
34 NET "O[3]" SLEW = FAST;
35 NET "O[4]" SLEW = FAST;
36 NET "O[5]" SLEW = FAST;
37 NET "O[6]" SLEW = FAST;
38

```

Design Summary (Programming File Generated) task2.v task2.ucf

Output:

